Resumen Técnico de Arquitectura: Hub de Incidencias Urbanas

Hack to the Future · Córdoba Tech Week · 12–13/11/2026 · Referencia: `arquitectura-sistema.md`, `contratos.md` y `decision-log.md`

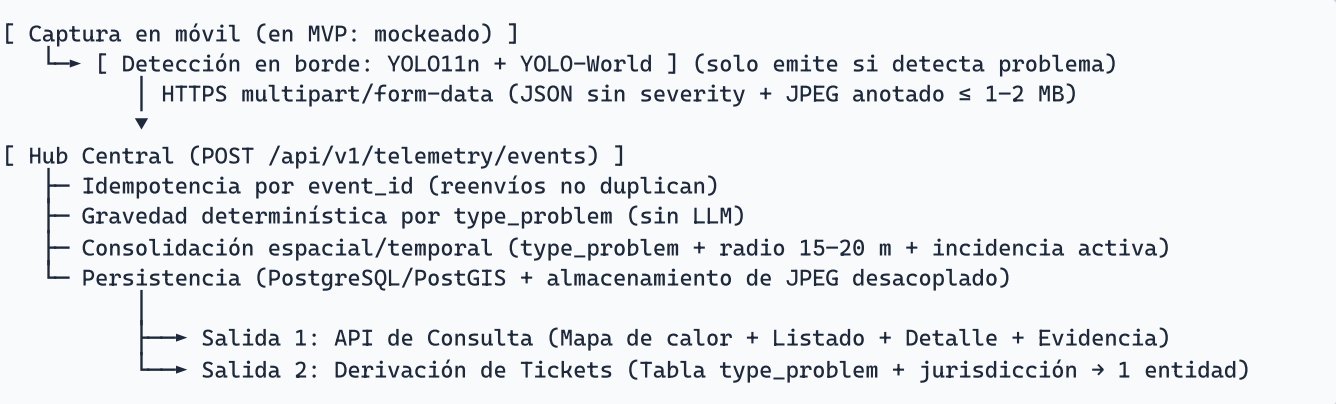
> **Alcance del MVP actual:** No incluye código ejecutable de backend ni inferencia en vivo; el MVP se presenta con **datos y flujos mockeados en el frontend** para validar el recorrido, los contratos y el modelo de decisión. Este documento resume la arquitectura del servicio objetivo para responder preguntas técnicas.

1. Qué es el servicio y qué resuelve

Es un **hub central de incidencias urbanas** que aprovecha el patrullaje vehicular existente (móviles municipales, policiales o flotas que ya recorren la ciudad) para detectar anomalías en la vía pública sin sumar kilómetros extra, consolidarlas en puntos únicos y alimentar dos salidas independientes:

- **Salida 1 — Mapa analítico y panel de consulta:** visualización de demanda y toma de decisiones estratégicas.
- **Salida 2 — Derivación automática de tickets:** envío directo de avisos al área municipal competente.

2. Flujo de la arquitectura objetivo



3. Decisiones técnicas clave y justificación

- **Visión artificial en Edge (post-MVP):** Enfoque híbrido con **YOLO11n** (rápido, bajo consumo ~5–10 W, para clases con pesos disponibles como basura/contenedores y baches) + **YOLO-World** (*open-vocabulary* para clases sin dataset propio como `poste_caido`, `luminaria_apagada_rota`, `semaforo_danado`). Solo transmite ante detección positiva; nunca envía video continuo.
- **Transporte e idempotencia: HTTPS** `multipart/form-data` enviando JSON + JPEG anotado (con recuadro ya dibujado, sin pixelado, ≤ 1–2 MB) en una sola transacción. El hub confirma solo tras persistir ambos; el emisor retiene y reintenta conservando el mismo `event_id`. *(MQTT queda para integración futura con hardware real).*
- **Gravedad determinística (sin LLM):** El emisor no envía `severity`. El hub la asigna mediante reglas fijas por `type_problem` (predecible, auditable, sin latencia ni costo de LLM por evento):
  - extremo `poste_caido`, `semaforo_danado`
  - alto `contenedor_desbordado`, `bache_calzada`
  - medio `basura_acumulada_piso`, `luminaria_apagada_rota`, `contenedor_lleno`
  - bajo `contenedor_vacio` *(normal / sin despacho)*

- **Consolidación y conteos:** Une observaciones por `type_problem` + proximidad (`15–20 m`) sobre una incidencia `activa` (ventana de `6 h` solo para búsqueda de recientes; no crea una incidencia nueva al vencer si sigue activa). `observation_count` cuenta detecciones distintas; `pass_count` se **estima** en el hub agrupando por `device_id` e intervalo configurable (sin `pass_id` obligatorio en el emisor). El **mapa de calor** suma **1 contribución por incidencia activa** ponderada por gravedad, evitando inflar la demanda por cantidad de frames.
- **Jurisdicción y tickets:** La **jurisdicción** es el ámbito geográfico/administrativo calculado en el hub desde `(lat, lon)` mediante polígono o *bounding-box* simple (`cordoba_centro` , `cordoba_norte` , `cordoba_sur` , o `cordoba_capital` por defecto). El ruteo usa una tabla configurable `(type_problem, jurisdiccion) → recipient_id` (1 entidad por incidencia). El ticket lleva `ticket_id` estable y `evidence_ref` controlada (sin adjuntar binario).
- **Máquina de estados desacoplada:**
  - **Ticket:** `pendiente_envio | fallido | recibido | en_atencion | resuelto`
  - **Incidencia:** `activa | en_atencion | resuelta` *(a futuro:* `posiblemente_resuelta` *por cámaras).*
  - **Regla estricta:** recibir un ticket no resuelve el problema, un fallo de entrega no oculta la incidencia del mapa, y **nunca se cierra una incidencia por ausencia de nuevos eventos**.

4. Contratos de API diseñados

Endpoint	Rol	Qué devuelve
`POST /api/v1/telemetry/events`	Ingesta (`multipart/form-data` : `event` JSON + `image` JPEG)	`incident_id` , `evidence_id` , `deduplicated` (201/200 tras guardar ambos)
`GET /api/v1/analytics/heatmap`	Capa geográfica filtrable (tipo, gravedad, estado, entidad, fechas)	GeoJSON con 1 punto por incidencia activa ponderado por gravedad
`GET /api/v1/incidents`	Listado paginado con los mismos filtros	Lista de incidencias consolidadas (sin binarios)
`GET /api/v1/incidents/{incident_id}`	Detalle de una incidencia puntual	Datos completos + arreglo `evidence_ids`
`GET /api/v1/evidence/{evidence_id}`	Acceso controlado a evidencia visual	Binario `image/jpeg` anotado (desacoplado del disco/storage)
`PATCH /api/v1/tickets/{ticket_id}/status`	Actualización desde entidad receptora	Transición `recibido → en_atencion → resuelto`

5. Respuestas rápidas a preguntas técnicas frecuentes

¿Por qué no transmitir video continuo a la nube?
Por ancho de banda, costo celular y privacidad. Procesar en borde y enviar solo detecciones positivas (JSON + frame anotado) reduce drásticamente el tráfico y evita almacenar escenas sin incidentes.

¿Cómo evitan que 20 frames del mismo bache generen 20 tickets o inflen el mapa?
En dos capas: (1) idempotencia por `event_id` frente a reintentos de red, y (2) consolidación espacio-temporal (`type_problem` + radio `15–20 m` sobre incidencia `activa`), sumando `observation_count` y contando **1 contribución** en el mapa de calor.

¿Por qué no usar un LLM para decidir la gravedad?
Un LLM por evento agrega latencia, costo variable e inconsistencia. Reglas determinísticas por `type_problem` garantizan respuesta inmediata y total auditabilidad.

¿Qué ocurre si falla la entrega del ticket a un área municipal?
Las salidas están desacopladas: el ticket queda registrado como `pendiente_envio` o `fallido` conservando su `ticket_id` para reintentar, y la incidencia continúa visible en el mapa y listado.

¿Cómo garantizan que un problema no se cierre falsamente?
Nunca se cierra una incidencia por vencimiento de tiempo ni por ausencia de nuevas pasadas de patrulla; solo la entidad responsable (o verificación visual posterior) actualiza el estado.